Piano music continuation with Transformers

Mohammed Zahoor Mashahir - PES1UG24CS579
Mohammed Ibrahim - PES1UG24CS578
UE24CS352A Machine Learning | 13 October 2026

Our project generates a continuation of a solo-piano MIDI prompt. We use two versions of our Transformer: an initially trained model and a final model obtained after post-training. They share the same network architecture. The distinction is how their weights have been trained. A user supplies a musical starting idea, and the selected checkpoint predicts the events that follow it. The system exports the result as editable MIDI and piano audio.

Why a Transformer

Self-attention lets a musical event use earlier notes, chords and rhythmic patterns within the context. Causal masking hides future tokens, so training and generation both follow next-token prediction. This matches a continuation task in which the supplied prompt is the beginning of the sequence [1].

Architecture

Piano MIDI becomes pitch/velocity, onset and duration tokens on a 10 ms timing grid. A 17,727-entry vocabulary feeds 512-dimensional embeddings and eight decoder blocks with eight attention heads each. RoPE encodes token order in queries and keys [2]. Each block uses Pre-LayerNorm, residual connections, dropout 0.1 and a SwiGLU feed-forward layer of width 1,536 [3,4]. Final normalization and a tied output head produce next-token scores. Shared embedding/output weights save parameters. The 2,048-token context and 36.36M parameters balance modeling capacity with a 12 GB GPU.

Initial model

The initial model learns from MAESTRO piano performances with separate data splits. Its weights start randomly. Inputs and targets shift by one token, and cross-entropy teaches the model to predict the next event token. Piece-specific crops preserve complete note groups. Padding and supplied instrument/start tokens do not contribute to loss. AdamW, BF16 and gradient accumulation support training on our hardware.

Final model after post-training

The final model continues from the initially trained weights while keeping the same architecture. Post-training combines whole-passage pitch/time augmentation with continuation-focused examples. A shared pitch shift preserves intervals and chords; a shared time scale transforms onsets and durations together. Prompt tokens provide context while the loss scores the following music. Ordinary prediction examples remain in the mix. Smaller learning-rate updates adapt the learned weights to the prompt-continuation task.

Generation and contributions

Either checkpoint receives the tokenized prompt and generates one token at a time. Sampling controls variation and event constraints keep note groups valid. Generated tokens return as context until EOS, the onset horizon or the context limit. Decoded MIDI supplies the notes; a fixed renderer produces the piano audio.

Equal allocation: Zahoor covers data, tokenization and Transformer training; Ibrahim covers generation, rendering, the application and documentation. Both share method interpretation and demo preparation. Development used an AI coding assistant.

References

[1] Huang et al., Music Transformer (1809.04281). [2] Su et al., RoFormer (2104.09864). [3] Xiong et al., Layer Normalization (2002.04745). [4] Shazeer, GLU Variants (2002.05202). Papers: arxiv.org. MAESTRO: magenta.tensorflow.org/datasets/maestro, CC BY-NC-SA 4.0.